
Group MLR-21 WingA4_771 Minor Assignment 2

Nitin Maheshwari (230711)
nitinm23@iitk.ac.in

Vidit Agarwal (231148)
viditag23@iitk.ac.in

Jeetraj Gupta (230507)
jeetrajg23@iitk.ac.in

Aaryan Tiwari (230023)
aaryant23@iitk.ac.in

Yugan Jain (231202)
yuganjain23@iitk.ac.in

1 Problem Abstract

This Assignment requires us to analyze the Gaussian Mixture Model (GMM) and do detailed calculations on how to train and test the model, implement this on the MNIST dataset, and analyze how accuracy varies with the number of components (K) for both complete and censored (missing pixel) images.

2 Tasks

2.1 Problem 1

Question

Give detailed calculations on how to train a generative multi-classification model where each class-conditional distribution i.e. $\mathbb{P}[\mathbf{x}|y = c]$ is modeled as a mixture of K Gaussians

$$\mathbb{P}[\mathbf{x}|y = c] = \text{GMM}(\{\mathcal{N}(\boldsymbol{\mu}_k^c, \boldsymbol{\Sigma}_k^c)\}_{k \in [K]}, \boldsymbol{\pi}^c), c \in [C]$$

where $\boldsymbol{\pi}^c \in \Delta^{K-1}$ is a K -dimensional histogram vector encoding the selection probability of each of the K components in the GMM for the c^{th} class. Note that when discussing GMMs in lecture slides, we often assumed the selection probability to be uniform i.e. $\boldsymbol{\pi}^c = [\frac{1}{K}, \frac{1}{K}, \dots, \frac{1}{K}]$. However, we should avoid making that assumption here.

Solution

Our aim is to train a **generative multi-class classification model** where each class distribution is represented by a **Gaussian Mixture Model (GMM)** with K components:

$$\mathbb{P}(\mathbf{x} | y = c) = \sum_{k=1}^K \pi_k^{(c)} \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k^{(c)}, \boldsymbol{\Sigma}_k^{(c)}), \quad c \in [C]$$

The overall model parameters are

$$\Theta = \left\{ \phi_c, \theta^{(c)} = \{\pi_k^{(c)}, \boldsymbol{\mu}_k^{(c)}, \boldsymbol{\Sigma}_k^{(c)}\}_{k=1}^K \right\}_{c=1}^C,$$

where $\phi_c = p(y = c)$ denotes the class prior.

Step 1: Estimating Class Priors

Given the labeled dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, let N_c be the number of samples in class c . The maximum likelihood is

$$\phi_c = \frac{N_c}{N}$$

Step 2: Training a GMM for Each Class

For each class c , we maximize the log-likelihood of its data subset: $\mathcal{D}_c = \{\mathbf{x}_i : y_i = c\}$:

$$\mathcal{L}(\theta^{(c)}) = \sum_{\mathbf{x}_i \in \mathcal{D}_c} \log \left(\sum_{k=1}^K \pi_k^{(c)} \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k^{(c)}, \boldsymbol{\Sigma}_k^{(c)}) \right)$$

Because the log of a sum makes direct maximization difficult, we use the **Expectation–Maximization (EM) algorithm**.

Initialization

For each class c , initialize $\theta^{(c)} = \{\pi_k^{(c)}, \boldsymbol{\mu}_k^{(c)}, \boldsymbol{\Sigma}_k^{(c)}\}$. To avoid poor local optima, perform multiple random restarts of EM and retain the parameters that yield the highest final log-likelihood.

EM Algorithm

For each class c , iterate between the following steps until convergence.

E-Step (Expectation): Compute the fact that component k generated sample $\mathbf{x}_i$:

$$\gamma_{ik}^{(c)} = p(z_i = k \mid \mathbf{x}_i, y_i = c; \theta^{(t)}) = \frac{\pi_k^{(c)(t)} \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k^{(c)(t)}, \boldsymbol{\Sigma}_k^{(c)(t)})}{\sum_{j=1}^K \pi_j^{(c)(t)} \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j^{(c)(t)}, \boldsymbol{\Sigma}_j^{(c)(t)})}$$

Numerical Stability: To keep the calculations numerically stable, we first compute

$$\ell_{ik} = \log \pi_k^{(c)(t)} + \log \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k^{(c)(t)}, \boldsymbol{\Sigma}_k^{(c)(t)}), \quad m_i = \max_k \ell_{ik}$$

and then normalize:

$$\gamma_{ik}^{(c)} = \frac{\exp(\ell_{ik} - m_i)}{\sum_{j=1}^K \exp(\ell_{ij} - m_i)}$$

M-Step (Maximization): Compute the number of samples assigned to each component:

$$N_k^{(c)} = \sum_{\mathbf{x}_i \in \mathcal{D}_c} \gamma_{ik}^{(c)}$$

and update parameters:

$$\begin{aligned} \pi_k^{(c)(t+1)} &= \frac{N_k^{(c)}}{N_c}, & \boldsymbol{\mu}_k^{(c)(t+1)} &= \frac{1}{N_k^{(c)}} \sum_{\mathbf{x}_i \in \mathcal{D}_c} \gamma_{ik}^{(c)} \mathbf{x}_i, \\ \boldsymbol{\Sigma}_k^{(c)(t+1)} &= \frac{1}{N_k^{(c)}} \sum_{\mathbf{x}_i \in \mathcal{D}_c} \gamma_{ik}^{(c)} (\mathbf{x}_i - \boldsymbol{\mu}_k^{(c)(t+1)})(\mathbf{x}_i - \boldsymbol{\mu}_k^{(c)(t+1)})^\top \end{aligned}$$

Stop when convergence is achieved.

Summary of Algorithm.

1. For each class c , extract $\mathcal{D}_c = \{\mathbf{x}_i : y_i = c\}$.
2. Initialize parameters using K -means.
3. Repeat E-step and M-step until convergence of $\mathcal{L}(\theta^{(c)})$.
4. Compute class priors $\phi_c = N_c/N$.
5. Classify new samples by Bayes rule using learned GMMs.

Note: We learn non-uniform mixture weights directly from data. This allows the model to capture varying cluster sizes within each class, resulting in a more flexible and realistic generative classifier.

2.2 Problem 2

Question

Give detailed calculations on how can inference (testing) be performed with such a GMM based generative multi-classification model on a new test point. (10 marks)

This section details the inference procedure for a new test point $\mathbf{x}$, utilizing the parameters $\hat{\Theta} = \{\hat{\phi}_c, \hat{\theta}^{(c)}\}_{c=1}^C$ estimated during training, where $\hat{\theta}^{(c)} = \{\hat{\pi}_k^{(c)}, \hat{\mu}_k^{(c)}, \hat{\Sigma}_k^{(c)}\}_{k=1}^K$ are the parameters of the GMM for class c .

Solution

Classification Rule (Bayes Rule)

As a generative classifier, the prediction for a new input $\mathbf{x}$ is based on the MAP rule, derived from Bayes Theorem. We seek the class c that maximizes the posterior probability $p(y = c|\mathbf{x})$:

$$\hat{y} = \arg \max_{c \in [C]} p(y = c|\mathbf{x}) = \arg \max_{c \in [C]} \frac{p(\mathbf{x}|y = c)p(y = c)}{p(\mathbf{x})}$$

Since the evidence $p(\mathbf{x})$ is constant across all classes, the maximization simplifies to:

$$\hat{y} = \arg \max_{c \in [C]} p(\mathbf{x}|y = c) \cdot p(y = c)$$

Step 1: Calculate Class-Conditional Likelihood $p(\mathbf{x}|y = c)$

For each class c , the likelihood of observing $\mathbf{x}$ is computed using the trained Gaussian Mixture Model (GMM):

$$p(\mathbf{x}|y = c) = \sum_{k=1}^K \hat{\pi}_k^{(c)} \mathcal{N}(\mathbf{x}|\hat{\mu}_k^{(c)}, \hat{\Sigma}_k^{(c)})$$

The density $\mathcal{N}(\mathbf{x}|\hat{\mu}_k^{(c)}, \hat{\Sigma}_k^{(c)})$ is the D -dimensional multivariate Gaussian density function:

$$\mathcal{N}(\mathbf{x}|\hat{\mu}_k^{(c)}, \hat{\Sigma}_k^{(c)}) = \frac{1}{\sqrt{(2\pi)^D |\hat{\Sigma}_k^{(c)}|}} \exp \left(-\frac{1}{2} (\mathbf{x} - \hat{\mu}_k^{(c)})^T (\hat{\Sigma}_k^{(c)})^{-1} (\mathbf{x} - \hat{\mu}_k^{(c)}) \right)$$

Step 2: Calculate the Log-Posterior and Predict

To maintain numerical stability, we perform the calculation using log.

The log-posterior for class c is:

$$\log p(y = c, \mathbf{x}) = \log p(\mathbf{x}|y = c) + \log p(y = c)$$

We use the trained class prior $\hat{\phi}_c = p(y = c)$.

Log-Likelihood Calculation

1. Calculate the log-density for each component k in class c :

$$L_{ck} = \log \hat{\pi}_k^{(c)} + \log \mathcal{N}(\mathbf{x}|\hat{\mu}_k^{(c)}, \hat{\Sigma}_k^{(c)})$$

2. Combine these log-densities to find the log of the GMM sum (the log-likelihood $\log p(\mathbf{x}|y = c)$) using Log-Sum-Exp:

$$\begin{aligned} \log p(\mathbf{x}|y = c) &= \text{LogSumExp}_{k=1}^K (L_{ck}) \\ \log p(\mathbf{x}|y = c) &= M_c + \log \left(\sum_{k=1}^K \exp(L_{ck} - M_c) \right) \end{aligned}$$

where $M_c = \max_k L_{ck}$.

Final Prediction

The predicted class $\hat{y}$ is determined by maximizing the total log-posterior across all classes:

$$\hat{y} = \arg \max_{c \in [C]} \left(\log p(\mathbf{x}|y = c) + \log \hat{\phi}_c \right)$$

2.3 Problem 3 : Implementation and Analysis

Question

Extend the lecture code to perform generative classification on the MNIST dataset using GMMs for each digit instead of a single Gaussian. **Do not change the train/test splits.** Note that the lecture code is equivalent to using a GMM with $K = 1$ (i.e. a single component per GMM). **Describe major changes required to the code to do this implementation.** Show a curve on how **training accuracy** and **test accuracy** vary as K is varied to use small and larger values, specifically $K = 1, 2, 5, 10, 15, 20$ etc. The x-axis of the curve should be K and the y-axis should show train/test accuracy. For these experiments, we ensure that every class gets the same value of K i.e. if using $K = 5$, ensure that each one of the 10 GMMs corresponding to the 10 digit classes uses $K = 5$ components in its respective GMM. Note that the test accuracies on this part should be reported on uncensored test images. (15 marks)

Description of Code Changes

The original lecture code was developed to support $K > 1$. The main challenge was the high dimensionality ($D = 784$), which makes estimating covariance matrices (Σ_k^c) computationally expensive and prone to overfitting. The critical change was to use **diagonal covariance matrices**.

The major changes are:

1. A new function was created to implement the EM algorithm described in Problem 1.
 - **Initialization:** Uses K -Means++ (KMPPInitGreedy) to find initial means.
 - **Covariance:** Initialize all variances to the global variance of the data.
 - **E-Step:** Calls `logpdfdiagonal` to compute component likelihoods.
 - **M-Step:** The update for covariance was modified to only update the diagonal variances:
$$\sigma_k^2 = \sum_i \gamma_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)^2 / N_k.$$
2. `logpdfdiagonal(X, mu, var, ...)`: A new function to compute the log-pdf of a diagonal Gaussian. It uses the sum of log-variances for the log-determinant and element-wise division for the quadratic term.
3. `learnGMMClassConditionalDist(X, y, K)`: This function was modified to:
 - Handle the $K = 1$ case by calculating and storing the mean and diagonal variance.
 - For $K > 1$, call 'fitGMMEM' for each class to get the GMM parameters ($\pi_c, \boldsymbol{\mu}_c, \sigma_c^2$).
 - The 'model' object now stores an array of variances instead of data full of covariance matrices, saving us an extra dimension of storage.
4. `predictGMMClassScores(X, model, ...)`: This function was updated to call `logpdfdiagonal` instead of the original `logpdf`, passing it the vector of variances for each component.

Results: Accuracy vs. K (Uncensored)

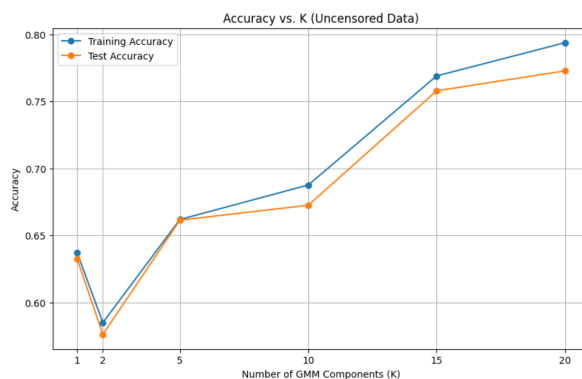


Figure 1: Training and Test Accuracy vs. Number of GMM Components (K) on uncensored data.

2.4 Problem 4 : Classification and Reconstruction on Censored Data

Question

Extend the lecture code to perform generative classification and reconstruction in the presence of missing pixels. **Do not change the manner in which pixels are omitted** i.e. do not change the method `tensorImages` that sets the window of missing pixels. **Describe major changes required to the code to do this implementation. Show a curve** on how **test accuracy** on images with missing pixels vary as K is varied to use small and larger values, specifically $K = 1, 2, 5, 10, 15, 20$ etc. **Also show example images** of successful and unsuccessful reconstructions and correct/incorrect classifications in the presence of missing pixels. Note that only test images are censored. Training images are not censored i.e. you should reuse the models you learnt in the previous part and simply re-test on censored images. (15 marks)

Description of Code Changes

1. **Censored Classification:** No major changes were needed for classification.
2. **Censored Reconstruction (`reconstructGMMImages(...)`):** The original `'reconstructImages'` function was completely replaced.
 - **Simplified Formula:** As we are using diagonal covariance matrices, the off-diagonal block Σ_{mo} is zero. This simplifies the conditional mean formula from $\mu_{m|o} = \mu_m + \Sigma_{mo}\Sigma_{oo}^{-1}(x_o - \mu_o)$ to simply $\mu_{m|o} = \mu_m$.
 - **Implementation:** The new function first computes the posterior weights $w_k(\mathbf{x}_o, c^*) = \mathbb{P}(z_k = 1 | \mathbf{x}_o, y = c^*)$ for the predicted class c^* , using `'logpdfdiagonal'` on the `*observed pixels* ( $\mathbf{x}_o$ )`.
 - The final reconstruction $\hat{\mathbf{x}}_m$ is then just the weighted average of the component means for the missing pixels: $\hat{\mathbf{x}}_m = \sum_{k=1}^K w_k(\mathbf{x}_o, c^*) \cdot \mu_{k,m}^{c^*}$.

Results: Accuracy vs. K (Censored)

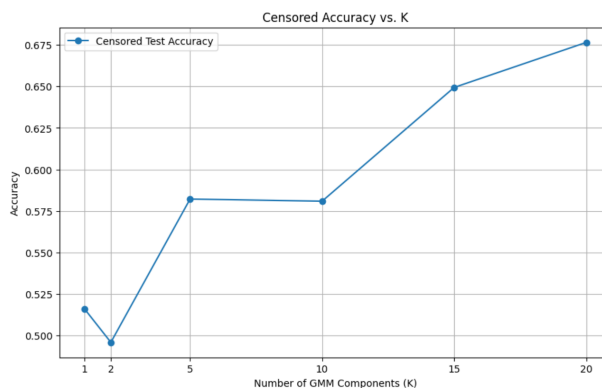
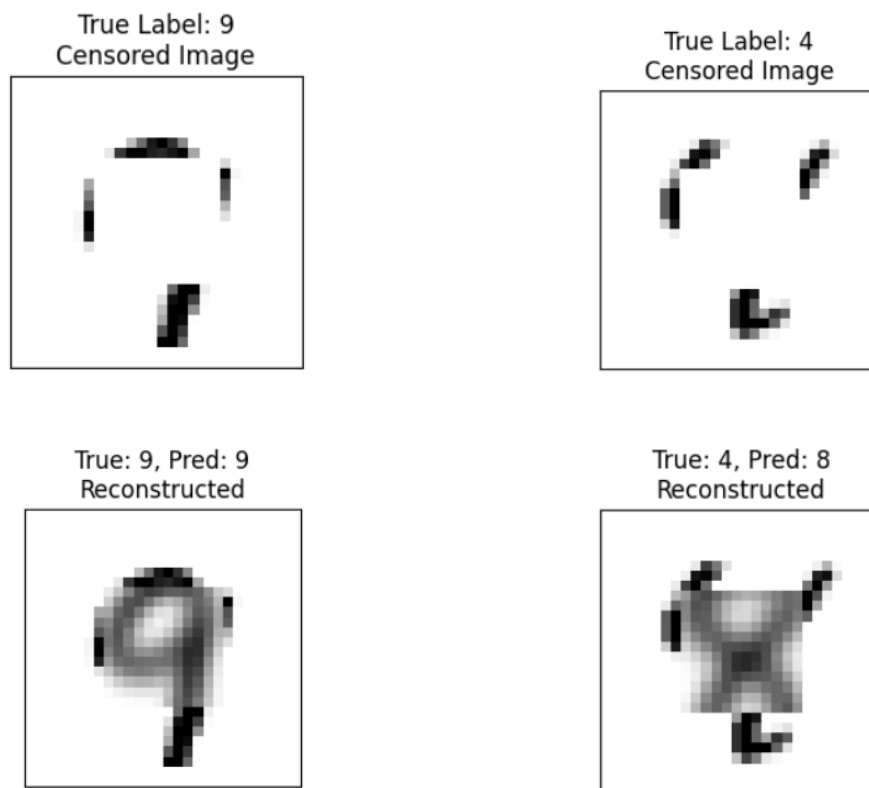


Figure 2: Test Accuracy vs. Number of GMM Components (K) on censored test images.

Example Reconstructions



(a) Successful reconstruction and classification

(b) Unsuccessful classification

Figure 4: The model's reconstruction of the images. In (a) the reconstruction is clearly a '9' and is classified correctly. In (b) the partial data for '4' is ambiguous and the model reconstructs a blurry shape resembling an '8', leading to a misclassification.

2.5 Part 5 : Bonus Question: Do Certain Classes Need Fewer GMMs?

Question

Do certain classes need fewer GMMs? Do certain digits exhibit more variations than others? Explore this by fixing a reasonably large value of K and instead of giving each class the same number of K components in its GMM, see if accuracy remains the same or similar even if certain classes are given fewer than K components. Which classes seem to require a large number of components? Which classes seem to be content with fewer components?

Answer

The bonus question explores the hypothesis that the intrinsic variability, or complexity, differs across the $C = 10$ digit classes ($c \in \{0, \dots, 9\}$). This variability must be captured by the number of Gaussian components (K_c) in each class-conditional GMM, $p(\mathbf{x}|y = c)$.

Hypothesis

We hypothesize that classes whose digits are written in a more standardized (simple) manner will require fewer components to model their distribution, while classes with more variation will require a larger number of components to maintain accuracy.

Complexity of Drawing Digits

1. **Low Complexity (Fewer K_c Needed):** Digits like '0' (generally an oval or circle), '1' (a vertical line) and '8' (two stacked circles) tend to have a highly constrained global structure.

2. **High Complexity (More K_c Needed):** Digits like 4', 7', and 9' often exhibit significant writing style variations. These variations will likely require a larger K to model all major clusters of data.

3 References

- Numerical Stability & LogSumExp Method
- LogSumExp Method
- GMMs
- Multi K-GMMs